Section 08: Embeddings, Vector Stores & Retrievers

LangChain v1.2.x (March 2026)

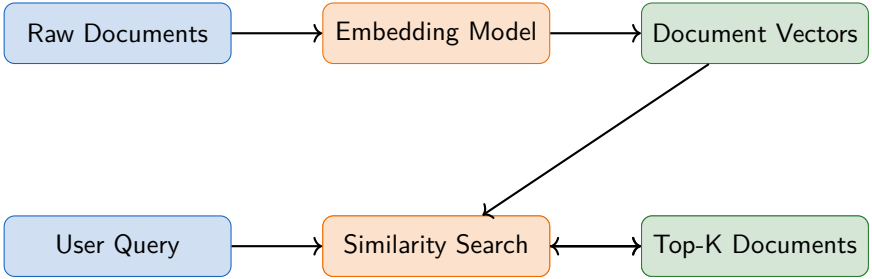
LangChain Documentation

Roadmap: Embeddings, Vector Stores & Retrievers

- **Mental Model:** The semantic search pipeline
- **Embeddings:** Converting text to vectors, similarity
- **The Embeddings Interface:** Core methods and providers
- **Vector Stores:** Purpose, implementations, operations
- **Retrievers:** Fetching relevant documents
- **Advanced Patterns:** Multi-query, compression, ensemble
- **Performance & Pitfalls:** Real-world considerations

Goal: Build production-ready retrieval pipelines

Mental Model: The Semantic Search Pipeline



Key Insight

Documents and queries are converted to vectors in the *same* embedding space.
Similarity = semantic relatedness in vector space.

What Are Embeddings? Text → Vectors

- **Embedding:** A dense vector representation of text
- Captures *semantic meaning*, not just bag-of-words
- Example dimensions: 384 (small), 1536 (OpenAI text-embedding-3-small), 3072 (text-embedding-3-large)

"A cat is sleeping on the mat"

$-0.15, 0.88, \ldots, 0.23]$

Semantic Similarity

"A cat rests on the floor" produces a *nearby* vector → high cosine similarity.
"The weather is sunny today" produces a *distant* vector → low cosine similarity.

The Embeddings Interface: Core Methods

API Contract

All embedding providers implement the same interface:

```python
from langchain_core.embeddings import Embeddings

class Embeddings(ABC):
    """Abstract base for embedding models."""

    @abstractmethod
    def embed_documents(
        self, texts: List[str]
    ) -> List[List[float]]:
        """Embed a list of documents."""
        pass

    @abstractmethod
    def embed_query(self, text: str) -> List[float]:
        """Embed a single query string."""
        pass
```

[Docs: langchain-core.embeddings]

OpenAIEmbeddings Setup and Usage

```python
import os
from langchain_openai import OpenAIEmbeddings

# Initialize with OPENAI_API_KEY environment variable
embeddings = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

# Embed documents (batch)
docs = [
    "LangChain simplifies LLM application development",
    "Embeddings map text to vectors"
]
doc_vectors = embeddings.embed_documents(docs)
print(f"Doc vectors: {len(doc_vectors)} x {len(doc_vectors[0])}")

# Embed a single query
query = "What is semantic search?"
query_vector = embeddings.embed_query(query)
print(f"Query vector dimension: {len(query_vector)}")
```

[Docs: langchain-openai.embeddings]

Other Embedding Providers

Anthropic Embeddings:

```
from langchain_anthropic import \
  AnthropicEmbeddings
emb = AnthropicEmbeddings(
  model="claude-3-5-sonnet-20241022"
)
```

Ollama (local):

```
from langchain_ollama import OllamaEmbeddings
emb = OllamaEmbeddings(
  model="nomic-embed-text"
)
```

HuggingFace (remote):

```
from langchain_huggingface import \
  HuggingFaceEmbeddings
emb = HuggingFaceEmbeddings(
  model_name="all-MiniLM-L6-v2"
)
```

Cohere:

```
from langchain_cohere import CohereEmbeddings
emb = CohereEmbeddings(
  model="embed-english-v3.0"
)
```

Note

Local embeddings (HuggingFace, Ollama) = free but slower. Cloud = faster but costs per API call.

Comparison: Embedding Models (March 2026)

Model	Provider	Dim	Cost	Quality
text-embedding-3-small	OpenAI	1536	$0.02/1M	High
text-embedding-3-large	OpenAI	3072	$0.13/1M	Very High
all-MiniLM-L6-v2	HF (local)	384	Free	Medium
all-mpnet-base-v2	HF (local)	768	Free	High
nomic-embed-text-1.5	Ollama	768	Free	High
embed-english-v3.0	Cohere	1024	$0.10/1M	Very High
Claude-3.5-Sonnet	Anthropic	1024	$3/1M tokens	Very High

Selection Guide

Prototyping: all-MiniLM-L6-v2 (fast, free)
Production: text-embedding-3-small (quality/cost)
Privacy: Ollama local embeddings

What Are Vector Stores? Purpose & Design

Definition

A **vector store** is a database optimized for:

- Storing high-dimensional dense vectors
- Fast *approximate* nearest-neighbor (ANN) search
- Associating vectors with metadata (source, page, etc.)

How it works:

1. Index documents: convert texts to vectors, build search structure
2. Query: embed user query, find nearest vectors
3. Return: top-K matching documents with metadata

ANN vs Exact Search

Vector stores use *approximate* search (faster, slightly less accurate) not exhaustive search.

FAISS: In-Memory Vector Store for Prototyping

```python
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()

# Create vector store from documents
documents = [
    "LangChain enables RAG applications",
    "Embeddings enable semantic search",
    "Vector stores index documents efficiently"
]

vector_store = FAISS.from_texts(
    documents, embeddings
)

# Similarity search
results = vector_store.similarity_search(
    "semantic search", k=2
)
for doc in results:
    print(f"Content: {doc.page_content}")
```

Chroma: Persistent Local Vector Store

```python
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()

# Create/load persistent collection
vector_store = Chroma(
    collection_name="my_docs",
    embedding_function=embeddings,
    persist_directory="./chroma_db"  # Saved to disk
)

# Add documents with metadata
vector_store.add_documents([
    Document(
        page_content="...",
        metadata={"source": "doc1.pdf", "page": 5}
    )
])

# Search (persists across sessions)
results = vector_store.similarity_search("query")
```

Pinecone: Managed Cloud Vector Store

```python
from langchain_pinecone import PineconeVectorStore
from langchain_openai import OpenAIEmbeddings
import os

embeddings = OpenAIEmbeddings()

# Connect to Pinecone (requires API key)
vector_store = PineconeVectorStore(
    index_name="my-index",
    embedding=embeddings,
    namespace="production"
)

# Add documents (synced to Pinecone servers)
vector_store.add_documents(documents)

# Search across distributed index
results = vector_store.similarity_search(
    "machine learning", k=5
)
```

Pros: Fully managed, scales to billions, enterprise features. **Cons:** Monthly cost, vendor lock-in

Common Vector Store Operations

```
from langchain_chroma import Chroma

vector_store = Chroma(...)

# 1. Add documents
vector_store.add_documents(docs)

# 2. Add texts with explicit IDs
ids = vector_store.add_texts(
    ["text1", "text2"],
    ids=["id1", "id2"]
)

# 3. Similarity search (top-k)
results = vector_store.similarity_search(
    "query", k=3
)

# 4. Similarity search with scores
results_with_scores = \
  vector_store.similarity_search_with_score(
    "query", k=3
)
for doc, score in results_with_scores:
```

Vector Store as Retriever: as_retriever()

```python
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()
vector_store = Chroma(embedding_function=embeddings)
vector_store.add_documents(documents)

# Convert to retriever interface
retriever = vector_store.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3}
)

# Use in chains
from langchain.chains import create_retrieval_chain
from langchain_openai import ChatOpenAI

llm = ChatOpenAI()
chain = create_retrieval_chain(retriever, llm)

result = chain.invoke({"input": "What is RAG?"})
print(result["answer"])
```

[Docs: retrievers]

The Retriever Interface and How It Works

Retriever API

All retrievers implement one core method:

```
get_relevant_documents(query: str) -> List[Document]
```

Returns a list of documents ranked by relevance.

Key points:

1. Retrievers are *pluggable* — swap implementations without changing downstream code
2. Input: a string query
3. Output: ordered list of relevant `Document` objects
4. Used by chains, agents, and RAG systems to fetch context

Design Benefit

Same interface works for vector stores, BM25, web search, databases, etc.

Similarity Search vs MMR (Maximal Marginal Relevance)

Similarity Search: Returns documents most similar to query.

```
results = vector_store.similarity_search("topic", k=3)
# Returns: [doc_most_relevant, doc_2nd_relevant, doc_3rd_relevant]
```

Problem: Top results may be redundant (say the same thing).

MMR: Maximizes relevance *and* diversity.

```
retriever = vector_store.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 3, "fetch_k": 10}
)
results = retriever.get_relevant_documents("topic")
# fetch_k=10 candidates, return k=3 most diverse
```

When to Use MMR

Use MMR for synthesis tasks (survey, comparison) where diversity matters.
Use similarity search for direct QA where relevance is paramount.

Metadata Filtering in Retrievers

```python
from langchain_chroma import Chroma

vector_store = Chroma(...)

# Add documents with metadata
vector_store.add_documents([
    Document(
        page_content="...",
        metadata={"source": "arxiv.pdf", "year": 2024}
    ),
    Document(
        page_content="...",
        metadata={"source": "blog.md", "year": 2025}
    )
])

# Create retriever with metadata filter
retriever = vector_store.as_retriever(
    search_kwargs={
        "k": 3,
        "filter": {"year": 2025}  # Only 2025 docs
    }
)
```

Multi-Query Retriever: Generating Multiple Search Queries

```
from langchain.retrievers.multi_query import \
  MultiQueryRetriever
from langchain_chroma import Chroma
from langchain_openai import ChatOpenAI

vector_store = Chroma(...)
llm = ChatOpenAI()

# Wraps a base retriever
multi_query_retriever = MultiQueryRetriever.from_llm(
    retriever=vector_store.as_retriever(
      search_kwargs={"k": 2}
    ),
    llm=llm
)

# Generates 3 variants of the query
results = multi_query_retriever.get_relevant_documents(
    "What is embeddings?"
)
# Queries: "What is embeddings?", "How do embeddings work?", ...
```

Why Multi-Query?

Contextual Compression Retriever

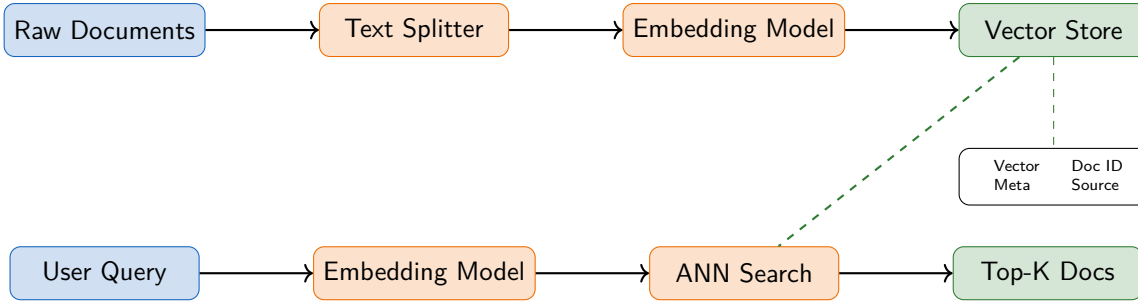
```
from langchain.retrievers import \
  ContextualCompressionRetriever
from langchain.retrievers.document_compressors \
  import LLMCompressor
from langchain_openai import ChatOpenAI
from langchain_chroma import Chroma

vector_store = Chroma(...)
llm = ChatOpenAI()

# Wraps base retriever with compression
compressor = LLMCompressor.from_llm(llm)
compression_retriever = \
  ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=vector_store.as_retriever(k=10)
  )

# Returns only relevant passages (not full docs)
results = compression_retriever.get_relevant_documents(
    "specific detail"
)
```

Ensemble Retriever: Combining Multiple Retrievers

```
from langchain.retrievers import EnsembleRetriever
from langchain_chroma import Chroma
from langchain_community.retrievers import BM25Retriever

documents = [...]

# Retriever 1: Vector store (semantic)
chroma = Chroma.from_documents(documents, embeddings)
vector_retriever = chroma.as_retriever(k=2)

# Retriever 2: BM25 (keyword-based)
bm25_retriever = BM25Retriever.from_documents(documents)
bm25_retriever.k = 2

# Ensemble: Reciprocal Rank Fusion
ensemble = EnsembleRetriever(
    retrievers=[vector_retriever, bm25_retriever],
    weights=[0.5, 0.5]  # Equal importance
)

results = ensemble.get_relevant_documents("query")
```

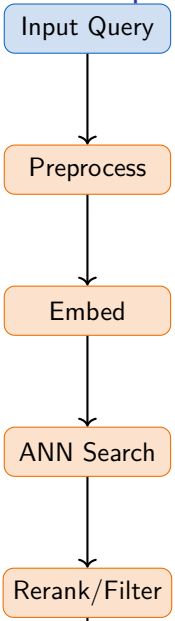
Why: Combines semantic + keyword strengths. More robust results.

Indexing Pipeline Visualization



Key: Documents and queries use the *same* embedding model.

Retrieval Pipeline in Detail



Performance Considerations: Index Size

- **Number of documents:** 1K (FAISS OK), 1M+ (Pinecone needed)
- **Vector dimensions:** Higher = more expressive but slower (1536 is practical maximum)
- **Search latency:** Trade-off between accuracy and speed
  - Exact search: O(n) — accurate but slow
  - HNSW/IVF (approximate): O(log n) — fast but lossy

Memory Cost

Storage = (num_docs × dimensions × 4 bytes) + metadata
Example: 1M docs × 1536 dims × 4 bytes = **6 GB**

Optimization Tips

Chunk documents (e.g., 300-token chunks) to balance coverage and retrieval precision.
Use quantization (8-bit) to reduce memory 4x at slight accuracy cost.

Performance Considerations: Embedding Cost

Pricing breakdown (OpenAI text-embedding-3-small):

- \$0.02 per 1 million tokens
- Indexing: 4000 words = 5000 tokens = \$0.0001
- For 100K documents: **\$10 one-time cost**

Query embedding cost:

- Each query = 100 tokens = \$0.000002
- Negligible at scale

Cost Optimization

Cache embeddings (don't re-embed the same documents).
Use free local embeddings (HuggingFace) if cost is critical.
Batch embedding calls for efficiency.

Performance Considerations: Search Latency

Typical latencies:

- **FAISS (in-memory):** 1–10 ms for 100K docs
- **Chroma (disk-backed):** 10–100 ms
- **Pinecone (cloud):** 100–500 ms (including network)

Optimization levers:

1. Reduce `k` (fetch fewer results)
2. Use quantization (trade accuracy for speed)
3. Shard index across multiple machines
4. Cache frequent queries

Production SLA

Target ¡ 200 ms end-to-end (embedding + search + LLM call).
Use ensemble of retrievers + background indexing for ¡100 ms.

Common Pitfall 1: Dimension Mismatch

Error Scenario

You index documents with embedding model A (1536 dims) but query with model B (384 dims).

Problem: Similarity computation fails or returns meaningless results.

Solution:

```
# WRONG: Different models for indexing and querying
index_embeddings = OpenAIEmbeddings(model="text-embedding-3-small")  # 1536
query_embeddings = HuggingFaceEmbeddings(model="all-MiniLM-L6-v2")  # 384

# RIGHT: Use same model consistently
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vector_store = Chroma(embedding_function=embeddings)
retriever = vector_store.as_retriever()
```

Best Practice

Define embedding model once, pass it to all components.

Common Pitfall 2: Missing or Incomplete Metadata

Error Scenario

You index documents without storing source information. Later, results include relevance scores but you don't know where they came from.

Solution:

```python
from langchain_core.documents import Document

# WRONG: No metadata
docs = [Document(page_content="...")]

# RIGHT: Include rich metadata
docs = [
    Document(
        page_content="...",
        metadata={
            "source": "arxiv:2310.04567",
            "page": 3,
            "date": "2023-10-01",
            "author": "Smith et al."
        }
    )
]
vector_store.add_documents(docs)
```

Common Pitfall 3: Wrong Similarity Metric

- **Cosine Similarity:** Standard (most vector stores), works for normalized embeddings
- **Euclidean Distance:** Less common, requires careful scaling
- **Dot Product:** Fast but requires pre-normalized vectors

Issue

If embeddings are `float32` unnormalized, Euclidean distance inflates with dimension count.

Best practice: Stick with cosine similarity (default in most libraries).

```
# Most vector stores default to cosine
results = vector_store.similarity_search(query, k=3)
# Equivalent to: cos(query_vec, doc_vec)
```

Note

OpenAI embeddings are normalized; cosine similarity equals dot product.
No need to manually normalize.

Common Pitfall 4: Ignoring Document Chunking

Problem

If you index entire PDFs (10K tokens each) without splitting, retrieval returns entire documents.
Context may be too large for LLM, includes irrelevant sections.

Solution: Chunk before indexing

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=50
)
chunks = splitter.split_documents(docs)
vector_store.add_documents(chunks)
```

Chunk Guidelines

Typical: 256–1024 tokens per chunk.
Overlap: 50–100 tokens (preserves context at boundaries).
Adjust based on domain (code needs longer chunks than prose).

Production Readiness Checklist

1. **Embedding Model:** Fixed and documented across all services
2. **Vector Store Persistence:** Backups, version control for index
3. **Metadata Completeness:** Every document has source, date, category
4. **Chunk Size Tuning:** Tested on representative corpus
5. **Search Quality:** Manual evaluation of top-K results
6. **Latency Monitoring:** Track p50, p95, p99 search times
7. **Cost Tracking:** Monitor embedding API calls and storage costs
8. **Freshness Strategy:** Plan for incremental indexing, re-indexing frequency
9. **Error Handling:** Graceful degradation if vector store is unavailable
10. **Security:** Secure API keys, data encryption at rest and in transit

[Docs: production-guide]

Real-World Example: RAG Chain with Retriever

```python
from langchain_chroma import Chroma
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

# Setup
embeddings = OpenAIEmbeddings()
vector_store = Chroma(embedding_function=embeddings)
vector_store.add_documents(my_documents)
retriever = vector_store.as_retriever(search_kwargs={"k": 3})

# Prompt template
system_prompt = (
    "You are a helpful assistant. "
    "Use context to answer: {context}"
)
prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("user", "{input}")
])

# Build chain
llm = ChatOpenAI()
chain = create_retrieval_chain(
    retriever,
    create_stuff_documents_chain(llm, prompt)
)

# Execute
result = chain.invoke({"input": "What is embeddings?"})
print(result["answer"])
```

Real-World Example: Hybrid Search (Vector + Keyword)

```python
from langchain.retrievers import EnsembleRetriever
from langchain_chroma import Chroma
from langchain_community.retrievers import BM25Retriever
from langchain_openai import OpenAIEmbeddings

# Vector retriever
embeddings = OpenAIEmbeddings()
vector_store = Chroma.from_documents(
    documents, embeddings
)
vector_retriever = vector_store.as_retriever(k=3)

# BM25 (keyword) retriever
bm25 = BM25Retriever.from_documents(documents)
bm25.k = 3

# Ensemble with equal weights
ensemble = EnsembleRetriever(
    retrievers=[vector_retriever, bm25],
    weights=[0.5, 0.5]
)

# Search uses both methods
results = ensemble.get_relevant_documents("ML model training")
# Semantically similar + keyword matches
```

Benefit: Handles both semantic and keyword queries robustly.

Advanced Pattern: Custom Retriever

```python
from langchain_core.retrievers import BaseRetriever
from langchain_core.documents import Document
from typing import List

class CustomDatabaseRetriever(BaseRetriever):
    """Retriever that queries a custom database."""

    def _get_relevant_documents(
        self, query: str
    ) -> List[Document]:
        # Custom logic: e.g., SQL query
        results = my_database.query(query)
        return [
            Document(
                page_content=row["text"],
                metadata={"id": row["id"]}
            )
            for row in results
        ]

# Use in chain
custom_retriever = CustomDatabaseRetriever()
chain = create_retrieval_chain(custom_retriever, llm)
```

Flexibility

Implement custom retrievers for databases, APIs, or hybrid logic.

Summary: Key Takeaways

1. **Embeddings** convert text to vectors in a shared semantic space
2. **Vector stores** enable fast similarity search for retrieval
3. **Retrievers** abstract vector stores behind a common interface
4. **Similarity search** is fast but MMR and ensemble methods add robustness
5. **Chunking, metadata, and consistency** are critical for production systems
6. **Costs scale** with document count (indexing) and query volume
7. **Advanced patterns** (compression, multi-query, ensemble) improve quality

RAG Pipeline Formula

Good Embeddings + Well-Chunked Documents + Robust Retrieval = High-Quality Context for LLM

Cheat Sheet: Quick Reference

- `OpenAIEmbeddings(model="text-embedding-3-small")` — Standard choice
- `Chroma(embedding_function=emb)` — Local persistent store
- `FAISS.from_texts(texts, emb)` — In-memory prototype
- `vs.as_retriever(search_type="mmr", search_kwargs={...})` — Convert to retriever
- `MultiQueryRetriever.from_llm(retriever, llm)` — Robustness boost
- `EnsembleRetriever(retrievers=[...], weights=[...])` — Hybrid search
- `ContextualCompressionRetriever(...)` — Reduce token usage

Debugging Tips

Print retrieved documents to check quality: `print([doc.page_content for doc in results])`
Check similarity scores: `similarity_search_with_score(query, k=3)`
Validate metadata filters applied: `print(doc.metadata)`

Self-Check Questions

1. How do embeddings enable semantic search? Why not just use keyword matching?
2. What is the difference between `embed_documents()` and `embed_query()`? Why have both?
3. Explain the trade-off between FAISS (in-memory) and Pinecone (managed cloud).
4. What does MMR mean and when would you use it instead of pure similarity search?
5. How does metadata filtering reduce false positives in retrieval?
6. Why is consistent chunking important for embeddings and retrieval?
7. Design a retrieval pipeline for a large document corpus (1M+ docs) with ¡ 200ms latency.

(See next slide for suggested answers)

Self-Check Answers (Brief)

1. Embeddings capture *semantic meaning* (synonyms, related concepts); keywords match only exact terms.
2. `embed_documents()` batches for efficiency; `embed_query()` signals intent (may be optimized differently).
3. FAISS: fast, free, single-machine; Pinecone: scalable, managed, costs money.
4. MMR adds diversity: useful for synthesis, summaries. Similarity search for direct QA.
5. Filters (e.g., by source, date) exclude irrelevant results early, reducing noise.
6. Consistent chunk size ensures stable semantic meaning and retrieval quality.
7. Use Pinecone or Redis, ensemble + filtering, quantization, caching, background indexing.

Further Reading & Resources

- LangChain Data Connection Guide
- Vector Store Integrations
- Embedding Providers
- Retrieval-Augmented Generation (RAG) survey
- DeepLearning.AI: Retrieval Augmented Generation
- FAISS GitHub: `https://github.com/facebookresearch/faiss`
- Chroma Docs: `https://docs.trychroma.com/`
- Pinecone Docs: `https://docs.pinecone.io/`

Next Steps

Build a small RAG prototype with Chroma + OpenAI embeddings.
Experiment with MMR, metadata filtering, and multi-query retrieval.
Benchmark performance on your domain.